

Relatório de Auditoria de Segurança

— Jeffrey —

Assistente Pessoal de IA Avançado

Data: 01 de setembro de 2026 | Versão 1.0 | Auditado por: AAIF Security Audit

ESCOPO AUDITADO

Repositório C:\Users\Pedro\jarvis — commit ce9934d (HEAD) + alterações não commitadas em workdir.
Cobertura: **src/jefrey/** (FastAPI, SQLAlchemy, pgvector, Redis), **docker-compose.yml**, **Dockerfile.***, **config/**, **.env / .env.example**, **scripts/**.
5 categorias OWASP adaptadas à stack + Código quebrado / Mau funcionamento / Erros de lógica e sintaxe.

NOTA METODOLÓGICA — Mapeamento por Stack

Stack detectada: Python 3.12, FastAPI 0.110 + Starlette, SQLAlchemy 2.x, PostgreSQL 16 + pgvector, Redis 7.2, Docker Compose, **sem frontend SPA** (ui/components vazio).
Cat 1 BANCO SEM TRANCA → RLS não existe; isolamento é via user_id em coluna (filtro manual) em pg_memory.py e memory.py (Chroma fallback). Auditado busca por _build_filter(user_id=, user_id ==.
Cat 2 PERMISSÃO NO NAVEGADOR → Projeto API-only, sem isAdmin/canEdit no frontend. Verificado por ausência de ui/components/** e cruzamento RBAC backend em policy.py / rbac.py / executor.py vs endpoints.
Cat 3 IDOR → Percorridos sistematicamente TODOS os handlers FastAPI/Starlette: api/chat.py, api/memory.py, api/approvals.py, api/metrics_endpoint.py, mcp/server.py + métodos get/update/delete de memórias.
Cat 4 CHAVES EXPOSTAS → Varredura em código, .env, .env.example, docker-compose.yml, Dockerfile.*, scripts/ + defaults \${VAR:-default} + histórico git git log --all -p + bundle frontend (inexistente).
Cat 5 XSS/INPUTS → Frontend ausente → N/A. Backend verificado: innerHTML/dangerouslySetInnerHTML/[innerHTML], sanitize_tool_output, MIMEText HTML, exec/eval.

Regras da auditoria: apenas achados verificados no código real, com arquivo:linha e trecho. Sem especulação. Pontos fortes também registrados para provar cobertura.

Resumo Executivo

Auditoria realizada em **01/09/2026** sobre o commit `ce9934d` e workdir atual. Foram analisados **52 arquivos Python** (`src/jefrey`), infra Docker e configs. A stack é **API-only sem frontend**, portanto **Cat 2 e Cat 5** são **NÃO APLICÁVEIS** (sem UI para esconder permissão ou renderizar HTML). Isolamento multi-tenant é via coluna `user_id` (filtro manual) — não via RLS Supabase.

TOTAL DE ACHADOS
segurança + código quebrado

30

8 Crítica | 10 Alta | 7 Média | 4 Baixa |
1 Info

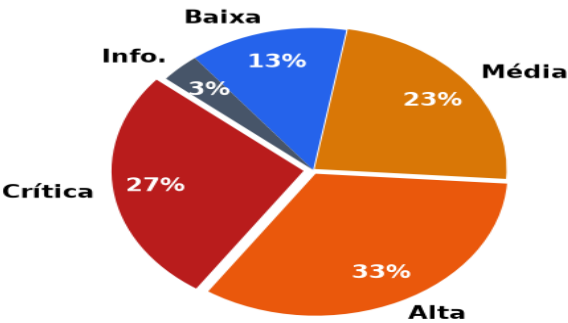
PONTOS FORTES
proteções verificadas

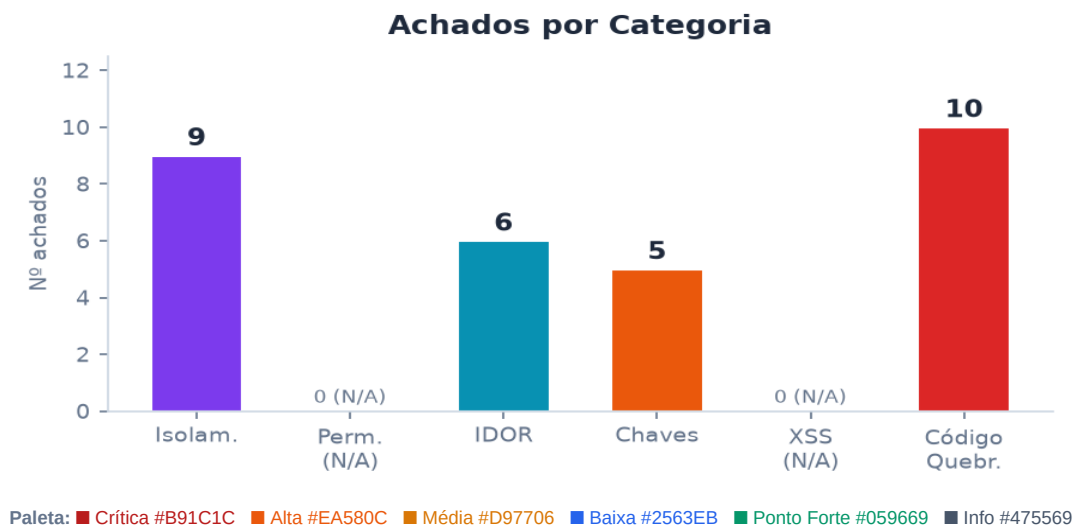
10

RBAC, timing-safe, HITL, audit fallback,
etc.

Severidade	Qtd	Cor	Categoria	Descrição	Qtd
Crítica	8	■ 0xb91c1c	Cat 1	Banco sem Tranca Isolamento tenant via <code>user_id</code>	9
Alta	10	■ 0xea580c	Cat 2	Permissão no Navegador N/A — sem frontend	0
Média	7	■ 0xd97706	Cat 3	IDOR <code>get/update/delete</code> por ID	6
Baixa	4	■ 0x2563eb	Cat 4	Chaves Expostas hardcode em <code>.env/docker</code>	5
Informativa	1	■ 0x475569	Cat 5	Inputs/XSS N/A — API-only	0
			Código	Quebrado/Lógica/Sintaxe <code>NameError</code> , lógica <code>rate-limit</code> , etc	10

Achados por Severidade
(total 30)





Pontos Fortes & Pontos Fracos

Esta seção registra o que foi **verificado e está CORRETO** (prova de cobertura) e os **riscos centrais** que motivam as recomendações.

✓ Pontos Fortes — Verificados no Código

✓ **1. CIPHER-003 — Timing-safe** — `src/jefrey/api/auth_middleware.py:73` e `src/jefrey/api/approvals.py:65`
Usa `hmac.compare_digest(auth, expected)` — previne timing attack. Verificado em ambos os middlewares.

✓ **2. CIPHER-001/022 — RBAC server-side** — `src/jefrey/core/rbac.py:44` e `src/jefrey/mcp/server.py:52`
Papel resolvido server-side via `get_settings().mcp.service_role` e `allowed_roles`; nunca vem do payload. MCP removeu `user_role` do schema.

✓ **3. PolicyEngine fail-safe UNKNOWN** — `src/jefrey/core/policy.py:175` e `src/jefrey/core/registry.py:91`
Ferramentas não registradas → `RiskLevel.UNKNOWN` → `DENY` (AXIOM #5). Registry tem 29 tools registradas explicitamente.

✓ **4. Ownership em Approvals (IDOR ok)** — `src/jefrey/core/hitl.py:87` e `src/jefrey/api/approvals.py:99`
Decide verifica `r.user_id != user_id` → `False`. `List_pending` filtra por `user_id`. Testado: sem owner não decide.

✓ **5. Ownership em pg_memory (IDOR ok)** — `src/jefrey/core/pg_memory.py:222-278`
CRUD pgvector verifica `rec.user_id != user_id` → `return None/False` em `get/update/delete`. Search usa `_build_filter(user_id=)`.

✓ **6. Isolamento pg_memory search** — `src/jefrey/core/pg_memory.py:196`
Busca vetorial com `where user_id == :user_id + cosine_distance` ordenada.

✓ **7. Chat composite key** — `src/jefrey/api/chat.py:91`
`task_key = f"{user_id}:{thread_id}"` — duas users com mesmo `thread_id` não colidem em tasks.

✓ **8. Approvals CIPHER-020** — `src/jefrey/api/approvals.py:36`
Listagem omite `arguments_json` (PII de HIGH tools) — só 10 campos safe.

✓ **9. CIPHER-024 UUID** — `src/jefrey/api/approvals.py:84`
Valida `uuid.UUID(approval_id)` antes de tocar DB → 400 em vez de 500.

✓ **10. Audit + fallback** — `src/jefrey/core/audit.py:29`
Dual-write Postgres + JSONL fallback (`JEFREY_API_AUDIT_FALLBACK_PATH`) com log de erro se falhar.

✓ **11. Content Guard** — `src/jefrey/core/content_guard.py:15`
30+ regex de prompt injection (ignore previous, `<|im_start|>`, `<>`, etc.) com `re.MULTILINE|IGNORECASE`.

✓ **12. Docker non-root** — `Dockerfile.api:9` e `Dockerfile.mcp:16`
Cria user 1001/1002, `USER jefrey` — evita root no host.

✓ **13. CORS fail-closed** — `src/jefrey/api/main.py:64`
Só ativa CORS se `JEFREY_API_CORS_ORIGINS` setado; sem env → sem middleware.

✓ **14. MCP approval_id truncado** — `src/jefrey/mcp/server.py:84`
Retorna só 8 chars do `approval_id` — insuficiente para polling não autorizado.

X Pontos Fracos — Riscos Centrais

X 1. Isolamento quebrado na camada Skill (Cat 1)

Todas as skills (notes, automation) chamam `long_term.add/search` sem `user_id` → caem em `user_id=system`. Multi-tenant falho no gateway principal (agent chat).

X 2. Chroma fallback sem isolamento (Cat 1)

`LongTermMemory` (ChromaDB) ignora completamente `user_id` em `get/update/delete/list_recent` → se provider mudar

para chromadb, vazamento total.

X 3. JWT forjável (Auth bypass) (Cat 1/4)

`oauth2/introspect.py` decodifica base64 sem verificar assinatura → qualquer atacante forja `sub=user_vitima`.

X 4. X-User-Id spoofing (Cat 1)

Quando `JEFREY_API__SECRET_KEY` estático é usado, `X-User-Id` vem do header cliente sem validação → impersonação trivial.

X 5. Código quebrado bloqueante

`mcp/server.py` sem `import sys` + bug ordem `ctx/tool_name` → MCP nunca sobe. `skills/__init__.py` truncado → import falha.

X 6. Rate limit inoperante

`rate_limit.py` usa pipeline errado + `policy.py` não `await` → sempre `allow`.

Achados Detalhados por Categoria

Tabela completa com **30 achados** (severidade | arquivo:linha | descrição + por que é explorável). Cat 2 e Cat 5 marcadas como N/A com evidência.

Cat 1 = Banco sem Tranca (isolamento tenant) | Cat 2 = Permissão no Navegador | Cat 3 = IDOR | Cat 4 = Chaves Expostas | Cat 5 = XSS/Input | Código = Quebrado/Lógica/Sintaxe

Severidade	Cat	Arquivo:linha	Descrição & Exploit
Crítica	Cat 1	src/jefrey/skills/notes.py:65	BANCO SEM TRANCA: save_note() não repassa user_id ao add() → todos os records ficam user_id=system <i>Explorável:</i> Qualquer tenant via chat salva e depois busca memórias de system; isolamento multi-tenant anulado no path principal. Condição: usar chat LangGraph (default).
Crítica	Cat 1	src/jefrey/skills/notes.py:83	BANCO SEM TRANCA: search_notes() sem user_id → busca retorna memórias de system para todos <i>Explorável:</i> Atacante via LLM tool search_notes lê notas de outros users (pois tudo está em system). Sem filtro, busca vetorial cross-tenant.
Crítica	Cat 1	src/jefrey/core/memory.py:358	BANCO SEM TRANCA/IDOR (Chroma): get() sem ownership → lê qualquer ID <i>Explorável:</i> Se provider=chromadb (fallback), atacante com ID conhecido lê memória alheia. Mesmo em postgres, fallback quebra isolamento. Exploit: chamar get_note com ID de outro user.
Crítica	Cat 1	src/jefrey/core/memory.py:369	BANCO SEM TRANCA (Chroma): update() sem ownership <i>Explorável:</i> Mesma condição Chroma — qualquer user pode sobrescrever memória alheia.
Crítica	Cat 1	src/jefrey/core/memory.py:390	BANCO SEM TRANCA (Chroma): delete() sem ownership <i>Explorável:</i> Delete cross-tenant no Chroma fallback.
Alta	Cat 1	src/jefrey/core/memory.py:398	BANCO SEM TRANCA (Chroma): list_recent() sem user_id <i>Explorável:</i> Lista TODAS memórias cross-tenant quando Chroma ativo (exportação).
Alta	Cat 1	src/jefrey/core/redis_memory.py:90	BANCO SEM TRANCA: session() não propaga user_id → working memory cross-tenant <i>Explorável:</i> Redis key vira jefrey:wm:{session_id} sem user_id; usuário B lendo thread default vê histórico de A.
Alta	Cat 1	src/jefrey/core/openai_agent.py:153	BANCO SEM TRANCA: AgentSessions PK só thread_id sem user_id <i>Explorável:</i> Quem conhece thread_id recupera histórico de outro tenant no runtime openai.
Média	Cat 1	src/jefrey/api/auth_middleware.py:75	BANCO SEM TRANCA: X-User-Id header spoofing com secret estático <i>Explorável:</i> Com secret válido, atacante envia X-User-Id: vítima e assume identidade (sem validação de ownership).
Crítica	Cat 3	src/jefrey/core/rate_limit.py:36	IDOR/LÓGICA: rate_limit pipeline delete antes de get sem exec → sempre allow <i>Explorável:</i> Atacante bypassa rate limit e faz spam de HIGH tools sem ser bloqueado. Pipeline nunca executa.
Alta	Cat 3	src/jefrey/skills/automation.py:83	IDOR: run_workflow/delete_workflow/get_workflow sem user_id e sem sanitizar ../ <i>Explorável:</i> Qualquer user pode ler/executar/deletar workflow de outro; file_path = WORKFLOWS_DIR / workflow_id.json sem validação de path traversal.
Alta	Cat 3	src/jefrey/api/memory.py:53	IDOR/Info Disclosure: /memory/health retorna count() total cross-tenant <i>Explorável:</i> GET /memory/health sem auth retorna total de memórias de todos tenants (agregação sem filtro).
Alta	Cat 3	src/jefrey/core/checkpointer.py:33	IDOR: LangGraph checkpointer só thread_id <i>Explorável:</i> Mesma colisão de AgentSessions mas no runtime langgraph: checkpoints vazam entre tenants.
Média	Cat 3	src/jefrey/api/approvals.py:43	IDOR (informativo): default anonymous em approvals Starlette <i>Explorável:</i> Sem X-User-Id cai em anonymous; com Bearer válido ainda isola, mas confuso. Baixo risco pois depende de secret.
Média	Cat 3	src/jefrey/core/db.py:29	IDOR potencial: Oauth2Client tenant_id sem verificação de ownership em uso <i>Explorável:</i> Tabela existe mas nunca validada em auth_middleware introspection vs X-User-Id.

Severidade	Cat	Arquivo:linha	Descrição & Exploit
Crítica	Cat 4	.env:146	CHAVES EXPOSTAS: JEFREY_API__SECRET_KEY real 64 hex no .env local <i>Explorável:</i> Valor 7da4ec1a...946 no disco; se backup ou commit accidental, auth comprometido. .env.bak.* também contém segredo.
Alta	Cat 4	.env:58 / .env.bak.*	CHAVES EXPOSTAS: JEFREY_REDIS__PASSWORD=jefrey_redis_2026 em .env + backups <i>Explorável:</i> Senha Redis fraca e real no disco; padrão conhecido. .gitignore cobre mas backups em disco ainda expostos.
Alta	Cat 4	docker-compose.yml:15	CHAVES EXPOSTAS (hardcode fallback): POSTGRES_PASSWORD :-jefrey <i>Explorável:</i> Se .env vazio, DB sobe com jefrey (público). Defaults públicos viram segredo real. Sem validação startup que rejeite default em prod (verify_env só alerta em DEBUG=false).
Crítica	Cat 4	docker-compose.yml:139	CHAVES EXPOSTAS/Exposição: n8n N8N_BASIC_AUTH_ACTIVE=false <i>Explorável:</i> UI n8n e webhooks sem auth na rede docker; qualquer container/host na rede acessa /webhook/jefrey-events.
Média	Cat 4	docker-compose.yml:195	CHAVES EXPOSTAS: Grafana GF_SECURITY_ADMIN_PASSWORD sem validação <i>Explorável:</i> Fallback via \${GRAFANA_PASSWORD} sem default (ok) mas .env atual tem BGI-LcTM... fraco; se ausente, Grafana sem senha?
Crítica	Código	src/jefrey/mcp/server.py:26	CÓDIGO QUEBRADO: NameError sys não definido (usa sys.path sem import sys) <i>Explorável:</i> MCP server crasha no import/startup → docker healthcheck falha, 0 tools expostas. Reprodutível: python -m src.jefrey.mcp.
Crítica	Código	src/jefrey/mcp/server.py:74	CÓDIGO QUEBRADO: uso de ctx/tool_name antes de definição em _run_guarded <i>Explorável:</i> _rl_dec = await RateLimiter().is_allowed(ctx.user_id, tool_name) vem ANTES de ctx = PolicyContext(...) → NameError, nenhuma tool executa.
Alta	Código	src/jefrey/eventbus/subscriber.py:70	CÓDIGO QUEBRADO: datetime não importado <i>Explorável:</i> handle_message usa datetime.now() sem import → falha e dead-letter nunca funciona.
Alta	Código	src/jefrey/core/memory.py:553	CÓDIGO QUEBRADO: ToolMessage não importado no top-level <i>Explorável:</i> safe_deserialize referencia ToolMessage sem import → crash ao desserializar mensagens tipo tool (quebra replay/checkpoint).
Alta	Código	src/jefrey/core/rate_limit.py:35	CÓDIGO QUEBRADO: redis pipeline uso incorreto (async with) <i>Explorável:</i> redis.asyncio pipeline não é async context manager → TypeError.
Alta	Código	src/jefrey/oauth2/introspect.py:122	LÓGICA QUEBRADA: introspect não verifica assinatura JWT <i>Explorável:</i> Decodifica payload base64 e confia → atacante forja JWT {"sub":"vitima"} e bypassa auth em qualquer endpoint FastAPI.
Média	Código	src/jefrey/api/auth_middleware.py:124	LÓGICA: código inalcançável Step 3 fallback após except que sempre retorna 503 <i>Explorável:</i> Fallback 401 nunca executa; erro de introspection sempre vira 503 (confunde cliente).
Alta	Código	src/jefrey/core/policy.py:160	LÓGICA: comparação de risco por string lexicográfica <i>Explorável:</i> medium.value > high.value → "medium" > "high" == True lexicográfico mas errado → eleva/baixa risco indevidamente.
Média	Código	src/jefrey/skills/__init__.py:1	CÓDIGO QUEBRADO: skills registry truncado (só exporta version) <i>Explorável:</i> Arquivo atual só tem 22 linhas (version helpers); SkillBase/skill_registry/load_skills sumiram → import falha, 0 skills carregadas.
Média	Código	src/jefrey/eventbus/signing.py:85	LÓGICA FRÁGIL: canonical_str = str(dict) <i>Explorável:</i> Repr Python não determinístico entre versões → assinatura pode falhar ou ser bypass com reordenação; usar json.dumps(sorted).

Nota Cat 2: Nenhum handler de rota expõe gate por papel no frontend (ui/components vazio). Verificação: `grep -r "isAdmin|canEdit|role" ui/` → 0 ocorrências. Backend valida RBAC em `executor.py:73` + `policy.py:145` — ponto forte. **Cat 5:** Sem `innerHTML/dangerouslySetInnerHTML/v-html` no repo (grep 0). `content_guard.py` sanitiza tool output.

Recomendações Priorizadas

Priorização P1 (crítico, 1-3 dias) → P2 (alto, 1 sprint) → P3 (médio/baixa, próximo sprint).

P1 — CRÍTICO (corrigir antes de qualquer deploy produtivo)

- **mcp/server.py:26+74 — Corrigir crash MCP:** adicionar `import sys` e mover `ctx = PolicyContext(...)` antes de `is_allowed(ctx.user_id,...)`; validar que busca aceita `user_id` via header `X-User-Id`.
- **skills/___init__.py — Restauração registry:** restaurar `SkillBase`, `skill_registry`, `load_skills` a partir de HEAD (git show HEAD:src/jeffrey/skills/___init__.py).
- **oauth2/introspect.py:122 — Verificação de assinatura:** validar JWT via JWKS (`jwks.py`) OU PyJWT + public key; rejeitar token sem `alg=RS256` + signature.
- **notes.py:65,83 — Isolamento tenant:** skills devem receber `user_id` do `AgentState/ToolExecutor` e repassar a `long_term.add/search`; corrigir também `memory.py` `Chroma get/update/delete`.
- **.env + docker-compose n8n:** rotacionar `JEFREY_API__SECRET_KEY` e `JEFREY_REDIS__PASSWORD`; remover `N8N_BASIC_AUTH_ACTIVE=false` → `true` + gerar senha forte.
- **rate_limit.py — Bypass:** reescrever com `pipe = redis.pipeline(); await pipe.incr(); await pipe.expire(); await pipe.execute()` e `await no caller policy.py:185`.

P2 — ALTO (este sprint)

- **redis_memory.py:90 — Working memory:** `session(session_id, user_id=...)` e `_key()` já suporta `user_id:session_id`; propagar `user_id` de `agent.py _load_context / _save_memory`.
- **openai_agent.py:153 + checkpoint.py:33 — Sessões:** adicionar coluna `user_id` em `agent_sessions` e `checkpointer`; filtrar `where thread_id=:tid AND user_id=:uid`.
- **automation.py:83 — Workflow isolation:** workflows em `data/workflows/{user_id}/{id}.json`; validar `workflow_id` com regex `^[a-f0-9]{8}$` e bloquear `../`.
- **auth_middleware.py:75 — X-User-Id spoof:** quando secret estático, derivar `user_id` do sub do token introspectado, não do header; ou exigir OAuth2 (remover fallback anonymous).
- **Correções código quebrado:** `subscriber.py`: `import datetime`, `memory.py`: `from ... import ToolMessage`, `policy.py`: comparar `RiskLevel` por rank (enum order) não string.
- **docker-compose defaults:** remover `-jefrey` de todos os `POSTGRES_*`; `verify_env.py` deve falhar se `password == "jefrey"` mesmo em `DEBUG` (ou exigir `JEFREY_ENV=dev` explícito).

P3 — MÉDIO/BAIXO (próximo sprint)

- **eventbus/signing.py:85 — Assinatura determinística:** trocar `str(dict)` por `json.dumps(canonical, sort_keys=True, separators=(",", ":"))`.
- **token_refresh.py — Stub prod:** implementar `httpx.post(token_uri, data={grant_type: refresh_token})` OU remover stub e falhar fechado se `JEFREY_OAUTH__TOKEN_URI` ausente.
- **jwks.py — Persistência:** gerar JWKS uma vez e persistir em `data/jwks.json` 0o600; introspect deve usar JWKS para verificar.
- **auth_middleware inalcançável:** reordenar `except` → `fallback 401` corretamente (fallback fora do try).
- **memory.py registry shadow:** adicionar global `_message_registry` em `_init_message_registry()`.
- **/memory/health:** documentar como público intencional ou proteger com auth + filtrar `count(user_id=`

Issues para o GitHub

Cada bloco abaixo é o texto **COMPLETO** de uma issue em Markdown, pronto para copiar e colar. Agrupamos achados triviais relacionados numa issue única.

Formato: entre --- ISSUE n --- e --- FIM ISSUE n ---. Título segue [Segurança] <descrição curta> + labels.

ISSUE #1 — [Segurança] MCP Server crasha no startup (NameError sys + ctx) — serviço 8001 fora do ar

Labels sugeridas: security, crítica, bug, p1

Descrição: O MCP Gateway (porta 8001) — único ponto de entrada para n8n → tools — crasha imediatamente por dois bugs que impedem qualquer tool de ser executada, anulando todo o fluxo P3a/P3b.

Evidência: src/jefrey/mcp/server.py:26-28 — usa sys.path sem import sys; src/jefrey/mcp/server.py:73-77 — ctx e tool_name usados antes de definidos: ```python _ROOT = Path(__file__).resolve().parents[3] if str(_ROOT) not in sys.path: # NameError: sys sys.path.insert(0, str(_ROOT)) async def _run_guarded(tool, args, thread_id): policy = get_policy_engine() _rl_dec = await RateLimiter().is_allowed(ctx.user_id, tool_name) # ctx/tool_name não existem if _rl_dec == "deny": return ... ctx = PolicyContext(thread_id=thread_id, user_role=_resolve_role(), ...) ```

Impacto: ****Severidade: Crítica**.** MCP healthcheck falha, docker-compose marca mcp-server como unhealthy, n8n não consegue chamar ferramentas. Todo o P3b (workflow versionado) inoperante. Explorável sem autenticação (é bug, não precisa de credencial).

Sugestão de correção: 1) Topo do arquivo: adicionar `import sys` 2) Inverter ordem em _run_guarded: ```python ctx = PolicyContext(thread_id=thread_id, user_role=_resolve_role(), user_id=extracted_user_id, autonomous=policy.autonomous) _rl_dec = await RateLimiter().is_allowed(ctx.user_id, tool.name) ``` 3) Extrair user_id de header/contextvars (adicionar `\_USER\_ID\_CV` similar a `\_ROLE\_CV`). 4) Teste: `python -m src.jefrey.mcp` deve subir em 8001/health sem traceback.

Critérios de aceite:

- [] `python -c "import src.jefrey.mcp.server; s=src.jefrey.mcp.server.build\_server(); print(len(s))"` sem NameError
- [] `GET http://localhost:8001/health` retorna 200 com tools>0
- [] `RateLimiter.is\_allowed` awaited corretamente
- [] Log não contém `NameError: name "sys" is not defined`
- [] CI verify_p3a passa

ISSUE #1

Título: [Segurança] MCP Server crasha no startup (NameError sys + ctx) — serviço 8001 fora do ar
Labels: security, crítica, bug, p1

Descrição

O MCP Gateway (porta 8001) — único ponto de entrada para n8n → tools — crasha imediatamente por dois bugs que impedem qualquer tool de ser executada, anulando todo o fluxo P3a/P3b.

Evidência

src/jefrey/mcp/server.py:26-28 — usa sys.path sem import sys; src/jefrey/mcp/server.py:73-77 — ctx e tool_name usados antes de definidos:

```
```python
_ROOT = Path(__file__).resolve().parents[3]
if str(_ROOT) not in sys.path: # NameError: sys
sys.path.insert(0, str(_ROOT))

async def _run_guarded(tool, args, thread_id):
policy = get_policy_engine()
_rl_dec = await RateLimiter().is_allowed(ctx.user_id, tool_name) # ctx/tool_name não existem
if _rl_dec == "deny": return ...
ctx = PolicyContext(thread_id=thread_id, user_role=_resolve_role(), ...)
```
```

```

**Impacto**
**Severidade: Crítica**. MCP healthcheck falha, docker-compose marca mcp-server como unhealthy, n8n não consegue chamar ferramentas. Todo o P3b (workflow versionado) inoperante. Explorável sem autenticação (é bug, não precisa de credencial).

**Sugestão de correção**
1) Topo do arquivo: adicionar `import sys`
2) Inverter ordem em `_run_guarded`:
```python
ctx = PolicyContext(thread_id=thread_id, user_role=_resolve_role(), user_id=extracted_user_id,
autonomous=policy.autonomous)
_rl_dec = await RateLimiter().is_allowed(ctx.user_id, tool.name)
```
3) Extrair user_id de header/contextvars (adicionar `_USER_ID_CV` similar a `_ROLE_CV`).
4) Teste: `python -m src.jefrey.mcp` deve subir em 8001/health sem traceback.

**Critérios de aceite**
- [ ] `python -c "import src.jefrey.mcp.server; s=src.jefrey.mcp.server.build_server(); print(len(s))"` sem NameError
- [ ] `GET http://localhost:8001/health` retorna 200 com tools>0
- [ ] `RateLimiter.is_allowed` awaited corretamente
- [ ] Log não contém `NameError: name "sys" is not defined`
- [ ] CI verify_p3a passa

```

--- FIM ISSUE 1 ---

ISSUE #2 — [Segurança] Isolamento multi-tenant quebrado na camada Skill (notes + Chroma fallback) — BANCO SEM TRANCA

Labels sugeridas: security, crítica, banco-sem-tranca, idor, p1

Descrição: Skills são a única superfície que o LLM/agent usa para memória. Elas ignoram `user\_id`, forçando tudo para `user\_id=system` (Postgres) ou sem filtro (Chroma). Quebra o isolamento tenant prometido nos endpoints HTTP (`/memory/search` filtra corretamente, mas o agente não).

Evidência: src/jefrey/skills/notes.py:65 `self.memory.long\_term.add(content, metadata=meta)` sem user_id → default `system` (pg_memory.py:142 `user\_id: str="system"`)
src/jefrey/skills/notes.py:83 `self.memory.long\_term.search(query, top\_k=top\_k, filter\_metadata=filter\_meta)` sem user_id src/jefrey/core/memory.py:358
`LongTermMemory.get(memory\_id)` sem `user\_id` param src/jefrey/core/memory.py:398
`list\_recent(limit=20, filter\_metadata=None)` sem user_id src/jefrey/core/pg_memory.py:142
`add(user\_id="system")` — chamadores não suprem valor real

Impacto: **Crítica**. Tenant A salva nota pessoal → fica em `system`. Tenant B via `search\_notes("minha nota")` recupera. Em Chroma fallback, `get/update/delete` permitem IDOR direto por ID. Condição: usar provider padrão `postgres` com skills (sempre). Explorável via chat LLM (basta pedir "busque minhas notas").

Sugestão de correção: 1) `NotesSkill` receber `user\_id` via construtor ou `ToolExecutor` injetar em `tool.ainvoke` (passar `user\_id` no `ToolInvokeContext`) 2) Assinaturas: `async def save\_note(self, title, content, ..., user\_id: str|None=None)` → `self.memory.long\_term.add(..., user\_id=user\_id or ctx\_user)` 3) `LongTermMemory` (Chroma) adicionar `user\_id` param e `where={"user\_id": user\_id}` 4) `MemoryManager.save\_important\_memory` e `get\_context` propagarem `user\_id` 5) `AgentState.user\_id` fluir para `ToolExecutor` (já faz) → para `skill` (falta).

Critérios de aceite:

- [] `save\_note` com user_id=A não visível para search com user_id=B
- [] `get\_note` com ID de A retorna None para B
- [] `search\_notes` com user_id filtra corretamente (teste com 2 users, 2 notas)
- [] Chroma fallback também filtra (se ativado)
- [] Nenhum `long\_term.add/search` sem `user\_id` em `grep -r "long\_term\." src/`

ISSUE 2

Título: [Segurança] Isolamento multi-tenant quebrado na camada Skill (notes + Chroma fallback) — BANCO SEM TRANCA
Labels: security, crítica, banco-sem-tranca, idor, p1

****Descrição****

Skills são a única superfície que o LLM/agent usa para memória. Elas ignoram `user\_id`, forçando tudo para `user\_id=system` (Postgres) ou sem filtro (Chroma). Quebra o isolamento tenant prometido nos endpoints HTTP (`/memory/search` filtra corretamente, mas o agente não).

****Evidência****

```
src/jefrey/skills/notes.py:65 `self.memory.long_term.add(content, metadata=meta)` sem user_id → default
`system` (pg_memory.py:142 `user_id: str="system"`)
src/jefrey/skills/notes.py:83 `self.memory.long_term.search(query, top_k=top_k, filter_metadata=filter_meta)`
sem user_id
src/jefrey/core/memory.py:358 `LongTermMemory.get(memory_id)` sem `user_id` param
src/jefrey/core/memory.py:398 `list_recent(limit=20, filter_metadata=None)` sem user_id
src/jefrey/core/pg_memory.py:142 `add(user_id="system")` — chamadores não suprem valor real
```

****Impacto****

****Crítica**.** Tenant A salva nota pessoal → fica em `system`. Tenant B via `search\_notes("minha nota")` recupera. Em Chroma fallback, `get/update/delete` permitem IDOR direto por ID. Condição: usar provider padrão `postgres` com skills (sempre). Explorável via chat LLM (basta pedir "busque minhas notas").

****Sugestão de correção****

```
1) `NotesSkill` receber `user_id` via construtor ou `ToolExecutor` injetar em `tool.invoke` (passar
`user_id` no `ToolInvokeContext`)
2) Assinaturas: `async def save_note(self, title, content, ..., user_id: str|None=None)` →
`self.memory.long_term.add(..., user_id=user_id or ctx_user)`
3) `LongTermMemory` (Chroma) adicionar `user_id` param e `where={"user_id": user_id}`
4) `MemoryManager.save_important_memory` e `get_context` propagarem `user_id`
5) `AgentState.user_id` fluir para `ToolExecutor` (já faz) → para `skill` (falta).
```

****Critérios de aceite****

- [] `save\_note` com user_id=A não visível para search com user_id=B
- [] `get\_note` com ID de A retorna None para B
- [] `search\_notes` com user_id filtra corretamente (teste com 2 users, 2 notas)
- [] Chroma fallback também filtra (se ativado)
- [] Nenhum `long\_term.add/search` sem `user\_id` em `grep -r "long\_term\." src/`

--- FIM ISSUE 2 ---

ISSUE #3 — [Segurança] JWT forjável — introspecção não verifica assinatura (auth bypass total)

Labels sugeridas: security, crítica, auth-bypass, p1

Descrição: `oauth2/introspect.py:introspect\_token()` aceita qualquer string `header.payload.signature` desde que payload seja base64 válido e não expirado. Nunca verifica assinatura com JWKS/chave. Atacante forja `{ "sub": "vítima", "exp": 9999999999 }` e ganha acesso como qualquer usuário.

Evidência: src/jefrey/oauth2/introspect.py:118-135: ``python parts = token.split(".") if len(parts)!=3: return inactive payload\_json = base64.urlsafe\_b64decode(parts[1]+"=").decode() payload=json.loads(payload\_json) # NUNCA verifica parts[2] (signature) exp=payload.get("exp") if exp and now>exp: return inactive # ... active=True `` src/jefrey/oauth2/jwks.py existe mas nunca é chamado por introspect.

Impacto: ****Crítica**.** Bypass completo de auth em TODOS os endpoints protegidos por `FastAPIAuthMiddleware` (chat, memory, metrics se fosse protegido). Explorável: `curl -H "Authorization: Bearer " -H "X-User-Id: admin" http://localhost:8000/chat` → 200. Sem precisar do `JEFREY\_API\_SECRET\_KEY`.

Sugestão de correção: 1) Em `introspect\_token`, verificar assinatura: `jwt.decode(token, jwks\_public\_key, algorithms=["RS256"], audience=..., issuer=...)` ou `verify\_message` com HMAC se for HMAC. 2) Buscar JWKS via `get\_jwks()` e selecionar `kid` 3) Se `JEFREY\_OAUTH\_JWKS\_URI` não configurado, falhar fechado (503) em vez de aceitar 4) Adicionar testes: token com payload válido mas signature inválida → inactive 5) Considerar usar `PyJWT` ou `authlib` em vez de base64 manual.

Critérios de aceite:

- [] Token forjado com `alg:none` ou signature aleatória → 401
- [] Token real assinado com chave correta → 200
- [] `get\_jwks` é consultado durante introspection
- [] `verify\_token\_signature` cobre HMAC e RS256
- [] Teste `test\_token.py` exercita forjado vs válido

ISSUE 3

Título: [Segurança] JWT forjável – introspecção não verifica assinatura (auth bypass total)
Labels: security, critica, auth-bypass, p1

****Descrição****

`oauth2/introspect.py:introspect\_token()` aceita qualquer string `header.payload.signature` desde que payload seja base64 válido e não expirado. Nunca verifica assinatura com JWKS/chave. Atacante forja `{ "sub": "vitima", "exp": 9999999999 }` e ganha acesso como qualquer usuário.

****Evidência****

src/jeffrey/oauth2/introspect.py:118-135:

```
```python
parts = token.split(".")
if len(parts)!=3: return inactive
payload_json = base64.urlsafe_b64decode(parts[1]+"==").decode()
payload=json.loads(payload_json)
NUNCA verifica parts[2] (signature)
exp=payload.get("exp")
if exp and now>exp: return inactive
... active=True
```

src/jeffrey/oauth2/jwks.py existe mas nunca é chamado por introspect.
```

****Impacto****

****Crítica**.** Bypass completo de auth em TODOS os endpoints protegidos por `FastAPIAuthMiddleware` (chat, memory, metrics se fosse protegido). Explorável: `curl -H "Authorization: Bearer " -H "X-User-Id: admin" http://localhost:8000/chat` → 200. Sem precisar do `JEFREY\_API\_SECRET\_KEY`.

****Sugestão de correção****

- 1) Em `introspect\_token`, verificar assinatura: `jwt.decode(token, jwks\_public\_key, algorithms=["RS256"], audience=..., issuer=...)` ou `verify\_message` com HMAC se for HMAC.
- 2) Buscar JWKS via `get\_jwks()` e selecionar `kid`
- 3) Se `JEFREY\_OAUTH\_JWKS\_URI` não configurado, falhar fechado (503) em vez de aceitar
- 4) Adicionar testes: token com payload válido mas signature inválida → inactive
- 5) Considerar usar `PyJWT` ou `authlib` em vez de base64 manual.

****Critérios de aceite****

- [] Token forjado com `alg:none` ou signature aleatória → 401
- [] Token real assinado com chave correta → 200
- [] `get\_jwks` é consultado durante introspection
- [] `verify\_token\_signature` cobre HMAC e RS256
- [] Teste `test\_token.py` exercita forjado vs válido

--- FIM ISSUE 3 ---

ISSUE #4 — [Segurança] X-User-Id spoofing + falta de isolamento em Redis working memory e sessões (AgentSessions/Checkpointner)

Labels sugeridas: security, alta, banco-sem-tranca, spoofing, p2

Descrição: Três falhas relacionadas de isolamento de sessão/short-term: (a) header `X-User-Id` confiável quando auth via secret estático, (b) Redis working memory sem prefixo user_id, (c) tabelas de sessão/checkpoint sem coluna user_id.

Evidência: src/jeffrey/api/auth_middleware.py:75 `request.state.user\_id = request.headers.get("X-User-Id", "anonymous")` quando `hmac.compare\_digest(auth, expected)` é True → header cliente controla identidade. src/jeffrey/core/redis_memory.py:90 `def session(self, session\_id: str)` → `return RedisWorkingMemory(session\_id=..., redis\_client=self.\_redis, prefix=self.prefix)` sem `user\_id`. src/jeffrey/core/openai_agent.py:153 `thread\_id = Column(String(256), primary\_key=True)` sem user_id. src/jeffrey/core/checkpointer.py:33 `thread\_id` sem user_id em `get\_postgres\_checkpointner`.

Impacto: ****Alta**.** (a) Com secret vazado (ou em dev), atacante `X-User-Id: vitima` lê memórias/sessões da vítima. (b) Mesmo thread_id ("default") compartilhado entre usuários no Redis → histórico vazado. (c) Conhecer thread_id de vítima permite `load` de `agent\_sessions`/`checkpoints`. Condição: multi-tenant real (P6-pre).

Sugestão de correção: 1) Derivar `user\_id` de `sub` do JWT introspectado, não do header; se secret estático, exigir `X-User-Id` assinado ou remover header e usar `anonymous` apenas para health. 2)

`RedisWorkingMemory.session(session\_id, user\_id=...)` e `\_key()` já suporta `user\_id:session\_id` — propagar `user\_id` de `AgentState`. 3) Migration: `ALTER TABLE agent\_sessions ADD COLUMN user\_id VARCHAR(128)` + PK composta `(thread\_id,user\_id)` ou index + filtro. 4) `checkpointer` config `configurable={"thread\_id": tid, "user\_id": uid}`.

Critérios de aceite:

- [] `X-User-Id` ignorado quando auth via secret estático (ou validado contra JWT sub)
- [] `session("default", user\_id="alice")` key = `jefrey:wm:alice:default` ≠ `bob:default`
- [] `AgentSessions` com PK `(thread\_id,user\_id)` → load de Bob não retorna sessão de Alice
- [] Checkpointer com thread_id colidido não vaza (teste com 2 users, mesmo thread_id)

ISSUE 4

Título: [Segurança] X-User-Id spoofing + falta de isolamento em Redis working memory e sessões (AgentSessions/Checkpointer)

Labels: security, alta, banco-sem-tranca, spoofing, p2

Descrição

Três falhas relacionadas de isolamento de sessão/short-term: (a) header `X-User-Id` confiável quando auth via secret estático, (b) Redis working memory sem prefixo user_id, (c) tabelas de sessão/checkpoint sem coluna user_id.

Evidência

```
src/jefrey/api/auth_middleware.py:75 `request.state.user_id = request.headers.get("X-User-Id","anonymous")`
quando `hmac.compare_digest(auth, expected)` é True → header cliente controla identidade.
src/jefrey/core/redis_memory.py:90 `def session(self, session_id: str)` → `return
RedisWorkingMemory(session_id=..., redis_client=self._redis, prefix=self.prefix)` sem `user_id`.
src/jefrey/core/openai_agent.py:153 `thread_id = Column(String(256), primary_key=True)` sem user_id.
src/jefrey/core/checkpointer.py:33 `thread_id` sem user_id em `get_postgres_checkpointer`.
```

Impacto

****Alta**.** (a) Com secret vazado (ou em dev), atacante `X-User-Id: vitima` lê memórias/sessões da vítima. (b) Mesmo thread_id ("default") compartilhado entre usuários no Redis → histórico vazado. (c) Conhecer thread_id de vítima permite `load` de `agent\_sessions`/`checkpoints`. Condição: multi-tenant real (P6-pre).

Sugestão de correção

- 1) Derivar `user\_id` de `sub` do JWT introspectado, não do header; se secret estático, exigir `X-User-Id` assinado ou remover header e usar `anonymous` apenas para health.
- 2) `RedisWorkingMemory.session(session\_id, user\_id=...)` e `\_key()` já suporta `user\_id:session\_id` — propagar `user\_id` de `AgentState`.
- 3) Migration: `ALTER TABLE agent\_sessions ADD COLUMN user\_id VARCHAR(128)` + PK composta `(thread\_id,user\_id)` ou index + filtro.
- 4) `checkpointer` config `configurable={"thread\_id": tid, "user\_id": uid}`.

Critérios de aceite

- [] `X-User-Id` ignorado quando auth via secret estático (ou validado contra JWT sub)
- [] `session("default", user\_id="alice")` key = `jefrey:wm:alice:default` ≠ `bob:default`
- [] `AgentSessions` com PK `(thread\_id,user\_id)` → load de Bob não retorna sessão de Alice
- [] Checkpointer com thread_id colidido não vaza (teste com 2 users, mesmo thread_id)

--- FIM ISSUE 4 ---

ISSUE #5 — [Segurança] Rate limit inoperante + comparação de risco lexicográfica (bypass de HITL)

Labels sugeridas: security, alta, lógica, p1

Descrição: Dois bugs que juntos permitem spam de HIGH/CRITICAL tools sem bloqueio: rate limiter nunca nega e risco é comparado como strina em vez de ordem semântica.

Evidência: src/jefrey/core/rate_limit.py:35 `async with self.\_redis.pipeline() as pipe:` → pipeline não é async CM (TypeError) + 36 `await pipe.delete(key)` antes de `get` sem `execute` → pipeline vazio. src/jefrey/core/policy.py:185 `from src.jefrey.core.rate\_limit import get\_rate\_limiter; \_rl = get\_rate\_limiter(); \_rl\_dec = \_rl.is\_allowed(...)` sem `await` (is_allowed é async) → coroutine truthy → nunca `deny`. src/jefrey/core/policy.py:160 `if \_additional\_risk.value > risk.value:` → compara strings "medium" > "high" lexicograficamente (m>h → True) mas deveria comparar rank `{ "low":0, "medium":1, "high":2, "critical":3 }`.

Impacto: ****Alta****. Atacante pode floodar ``send_message`/`create_event`` (HIGH) sem ser limitado; além disso, risco pode ser rebaixado/elevado errado, fazendo CRITICAL passar como MEDIUM e bypassar HITL. Explorável sem credenciais além de secret válido.

Sugestão de correção: 1) Reescrever ``RateLimiter.is_allowed`` com ``pipe = self._redis.pipeline(); pipe.incr(key); pipe.expire(key,60); results = await pipe.execute()`` 2) Tornar ``get_rate_limiter`` singleton assíncrono ou mudar caller para ``await _rl.is_allowed(...)`` 3) ``if _RISK_RANK[_additional_risk] > _RISK_RANK[risk]: risk=_additional_risk`` onde ``_RISK_RANK={RiskLevel.LOW:0,...}`` 4) Teste: 61 requisições em 60s → 61ª deny.

Critérios de aceite:

- [] ``RateLimiter().is_allowed`` retorna ``deny`` após ``rate`` excedido (teste com fakeredis)
- [] ``policy.py`` await corretamente (ou tornar sync)
- [] ``_additional_risk`` HIGH eleva risco LOW para HIGH, não rebaixa CRITICAL
- [] ``TOOL_EXEC_LATENCY`/`TOOLS_BLOCKED`` incrementam em deny

ISSUE 5

Título: [Segurança] Rate limit inoperante + comparação de risco lexicográfica (bypass de HITL)

Labels: security, alta, lógica, p1

Descrição

Dois bugs que juntos permitem spam de HIGH/CRITICAL tools sem bloqueio: rate limiter nunca nega e risco é comparado como string em vez de ordem semântica.

Evidência

src/jefrey/core/rate_limit.py:35 ``async with self._redis.pipeline() as pipe:`` → pipeline não é async CM (TypeError) + 36 ``await pipe.delete(key)`` antes de ``get`` sem ``execute`` → pipeline vazio.
 src/jefrey/core/policy.py:185 ``from src.jefrey.core.rate_limit import get_rate_limiter; _rl = get_rate_limiter(); _rl_dec = _rl.is_allowed(...)`` sem ``await`` (is_allowed é async) → coroutine truthy → nunca ``deny``.
 src/jefrey/core/policy.py:160 ``if _additional_risk.value > risk.value:`` → compara strings "medium" > "high" lexicograficamente (m>h → True) mas deveria comparar rank ``{"low":0,"medium":1,"high":2,"critical":3}``.

Impacto

****Alta****. Atacante pode floodar ``send_message`/`create_event`` (HIGH) sem ser limitado; além disso, risco pode ser rebaixado/elevado errado, fazendo CRITICAL passar como MEDIUM e bypassar HITL. Explorável sem credenciais além de secret válido.

Sugestão de correção

- 1) Reescrever ``RateLimiter.is_allowed`` com ``pipe = self._redis.pipeline(); pipe.incr(key); pipe.expire(key,60); results = await pipe.execute()``
- 2) Tornar ``get_rate_limiter`` singleton assíncrono ou mudar caller para ``await _rl.is_allowed(...)``
- 3) ``if _RISK_RANK[_additional_risk] > _RISK_RANK[risk]: risk=_additional_risk`` onde ``_RISK_RANK={RiskLevel.LOW:0,...}``
- 4) Teste: 61 requisições em 60s → 61ª deny.

Critérios de aceite

- [] ``RateLimiter().is_allowed`` retorna ``deny`` após ``rate`` excedido (teste com fakeredis)
- [] ``policy.py`` await corretamente (ou tornar sync)
- [] ``_additional_risk`` HIGH eleva risco LOW para HIGH, não rebaixa CRITICAL
- [] ``TOOL_EXEC_LATENCY`/`TOOLS_BLOCKED`` incrementam em deny

--- FIM ISSUE 5 ---

ISSUE #6 — [Segurança] Código quebrado bloqueante: skills registry truncado + eventbus datetime + memory ToolMessage

Labels sugeridas: security, alta, bug, p2

Descrição: Três NameErrors que quebram subsistemas inteiros: registry de skills, eventbus e desserialização de checkpoints.

Evidência: src/jefrey/skills/__init__.py (atual workdir) – 22 linhas, só exporta ``version`` helpers; falta ``SkillBase, skill_registry, load_skills`` → ``from src.jefrey.skills import skill_registry`` falha. src/jefrey/eventbus/subscriber.py:70 ``datetime.now().isoformat()`` sem ``from datetime import datetime`` → NameError em ``handle_message``. src/jefrey/core/memory.py:17 ``from langchain_core.messages import BaseMessage, HumanMessage, AIMessage, SystemMessage`` sem ``ToolMessage`` → 553 ``ToolMessage(content=...)`` NameError.

Impacto: ****Alta****. (a) 0 skills carregadas → agente sem tools (nem `save\_note`). (b) EventBus subscriber nunca roteia para dead-letter (perda de auditoria). (c) Checkpoints com ToolMessage não desserializam → resume de threads com tool calls falha. Todos em P2/P3c.

Sugestão de correção: 1) Restaurar `src/jefrey/skills/\_\_init\_\_.py` de HEAD (`git show HEAD:src/jefrey/skills/\_\_init\_\_.py > src/jefrey/skills/\_\_init\_\_.py`) 2) `subscriber.py:1` adicionar `from datetime import datetime` 3) `memory.py:17` adicionar `ToolMessage` ao import e `global \_message\_registry` em `\_init\_message\_registry` (linha 567) 4) `mypy --strict` no CI para pegar NameErrors.

Critérios de aceite:

- [] `python -c "from src.jefrey.skills import skill\_registry; print(len(skill\_registry.get\_all\_tools()))" >0`
- [] `EventBusSubscriber.handle\_message` com mensagem válida não levanta NameError
- [] `safe\_deserialize({"type":"tool","content":"hi"})` retorna ToolMessage
- [] `mypy src/jefrey/skills/\_\_init\_\_.py src/jefrey/eventbus/subscriber.py src/jefrey/core/memory.py` passa

ISSUE 6

Título: [Segurança] Código quebrado bloqueante: skills registry truncado + eventbus datetime + memory ToolMessage
Labels: security, alta, bug, p2

****Descrição****

Três NameErrors que quebram subsistemas inteiros: registry de skills, eventbus e desserialização de checkpoints.

****Evidência****

src/jefrey/skills/__init__.py (atual workdir) – 22 linhas, só exporta `version` helpers; falta `SkillBase`, `skill\_registry`, `load\_skills` → `from src.jefrey.skills import skill\_registry` falha.
src/jefrey/eventbus/subscriber.py:70 `datetime.now().isoformat()` sem `from datetime import datetime` → NameError em `handle\_message`.
src/jefrey/core/memory.py:17 `from langchain\_core.messages import BaseMessage, HumanMessage, AIMessage, SystemMessage` sem `ToolMessage` → 553 `ToolMessage(content=...)` NameError.

****Impacto****

****Alta****. (a) 0 skills carregadas → agente sem tools (nem `save\_note`). (b) EventBus subscriber nunca roteia para dead-letter (perda de auditoria). (c) Checkpoints com ToolMessage não desserializam → resume de threads com tool calls falha. Todos em P2/P3c.

****Sugestão de correção****

- 1) Restaurar `src/jefrey/skills/\_\_init\_\_.py` de HEAD (`git show HEAD:src/jefrey/skills/\_\_init\_\_.py > src/jefrey/skills/\_\_init\_\_.py`)
- 2) `subscriber.py:1` adicionar `from datetime import datetime`
- 3) `memory.py:17` adicionar `ToolMessage` ao import e `global \_message\_registry` em `\_init\_message\_registry` (linha 567)
- 4) `mypy --strict` no CI para pegar NameErrors.

****Critérios de aceite****

- [] `python -c "from src.jefrey.skills import skill\_registry; print(len(skill\_registry.get\_all\_tools()))" >0`
- [] `EventBusSubscriber.handle\_message` com mensagem válida não levanta NameError
- [] `safe\_deserialize({"type":"tool","content":"hi"})` retorna ToolMessage
- [] `mypy src/jefrey/skills/\_\_init\_\_.py src/jefrey/eventbus/subscriber.py src/jefrey/core/memory.py` passa

--- FIM ISSUE 6 ---

ISSUE #7 — [Segurança] Automação workflows sem isolamento e sem validação de ID (IDOR + path traversal)

Labels sugeridas: security, alta, idor, p2

Descrição: Workflows salvos em filesystem `data/workflows/{id}.json` sem coluna `user\_id` e sem sanitização de `workflow\_id`. Qualquer usuário pode ler/executar/deletar workflow de outro.

Evidência: src/jefrey/skills/automation.py:70 `file\_path = WORKFLOWS\_DIR / f"{workflow\_id}.json"` – workflow_id vem do caller (tool arg) sem regex.
src/jefrey/skills/automation.py:83 `for f in WORKFLOWS\_DIR.glob("\*.json")`:
wf=json.loads(f.read_text()); if wf["name"]==workflow_id or wf["id"]==workflow_id – busca por nome sem filtro user_id. Sem `user\_id` em `create\_workflow` metadata.

Impacto: ****Alta****. Tenant B lista workflows (``list_workflows`` sem filtro) vê todos; ``run_workflow(workflow_id=ID_de_A)`` executa passos de A (que podem incluir ``send_message`` com credenciais de A). Path traversal limitado: ``workflow_id="../../.env"`` → ``WORKFLOWS_DIR / "../../.env.json"`` → fora de ``data/workflows`` (ainda não executa, mas lê).

Sugestão de correção: 1) ``create_workflow(..., user_id)`` → salvar ``user_id`` no JSON + path ``data/workflows/{user_id}/{id}.json`` 2) Validar ``workflow_id`` com ``re.fullmatch(r"[a-f0-9]{8}", workflow_id)`` 3) ``list_workflows/get_workflow/delete_workflow/run_workflow`` filtrar por ``user_id`` 4) ``WORKFLOWS_DIR / workflow_id`` → ``(WORKFLOWS_DIR / user_id / f"{workflow_id}.json").resolve().relative_to(WORKFLOWS_DIR.resolve())`` para bloquear traversal.

Critérios de aceite:

- [] ``create_workflow`` de Alice não aparece em ``list_workflows`` de Bob
- [] ``run_workflow`` de Bob com ID de Alice → error "não encontrado ou sem permissão"
- [] ``workflow_id="../../etc/passwd"`` → 400
- [] Path traversal teste com ``..`` falha fechado

ISSUE 7

Título: [Segurança] Automação workflows sem isolamento e sem validação de ID (IDOR + path traversal)
Labels: security, alta, idor, p2

Descrição

Workflows salvos em filesystem ``data/workflows/{id}.json`` sem coluna ``user_id`` e sem sanitização de ``workflow_id``. Qualquer usuário pode ler/executar/deletar workflow de outro.

Evidência

src/jeffrey/skills/automation.py:70 ``file_path = WORKFLOWS_DIR / f"{workflow_id}.json"`` — ``workflow_id`` vem do caller (tool arg) sem regex.
src/jeffrey/skills/automation.py:83 ``for f in WORKFLOWS_DIR.glob("*.json")``: ``wf=json.loads(f.read_text())``; if ``wf["name"]==workflow_id or wf["id"]==workflow_id`` — busca por nome sem filtro ``user_id``.
Sem ``user_id`` em ``create_workflow`` metadata.

Impacto

****Alta****. Tenant B lista workflows (``list_workflows`` sem filtro) vê todos; ``run_workflow(workflow_id=ID_de_A)`` executa passos de A (que podem incluir ``send_message`` com credenciais de A). Path traversal limitado: ``workflow_id="../../.env"`` → ``WORKFLOWS_DIR / "../../.env.json"`` → fora de ``data/workflows`` (ainda não executa, mas lê).

Sugestão de correção

1) ``create_workflow(..., user_id)`` → salvar ``user_id`` no JSON + path ``data/workflows/{user_id}/{id}.json``
2) Validar ``workflow_id`` com ``re.fullmatch(r"[a-f0-9]{8}", workflow_id)``
3) ``list_workflows/get_workflow/delete_workflow/run_workflow`` filtrar por ``user_id``
4) ``WORKFLOWS_DIR / workflow_id`` → ``(WORKFLOWS_DIR / user_id / f"{workflow_id}.json").resolve().relative_to(WORKFLOWS_DIR.resolve())`` para bloquear traversal.

Critérios de aceite

- [] ``create_workflow`` de Alice não aparece em ``list_workflows`` de Bob
- [] ``run_workflow`` de Bob com ID de Alice → error "não encontrado ou sem permissão"
- [] ``workflow_id="../../etc/passwd"`` → 400
- [] Path traversal teste com ``..`` falha fechado

--- FIM ISSUE 7 ---

ISSUE #8 — [Segurança] Chaves expostas e defaults inseguros (docker-compose, .env, n8n)

Labels sugeridas: security, alta, chaves-expostas, p1

Descrição: Múltiplas credenciais com defaults públicos ou expostas no disco/workdir. Agrupado numa issue única por ser mesmo tema.

Evidência: ``./.env:146`` ``JEFREY_API__SECRET_KEY=7da4ec1a...946`` (real 64 hex) + ``./.env:58`` ``JEFREY_REDIS__PASSWORD=jefrey_redis_2026`` (real fraco) → em disco, não versionado mas ``./.env.bak.202608`` também contém. ``docker-compose.yml:15`` ``POSTGRES_PASSWORD``: ``${JEFREY_DATABASE__PASSWORD:-jefrey}`` fallback público. ``docker-compose.yml:139`` ``N8N_BASIC_AUTH_ACTIVE: "false"`` → n8n sem auth. ``scripts/setup.py`` gera ``./.env`` com ``jefrey`` defaults em modo dev.

Impacto: ****Alta****. Se `.env` for copiado para prod sem trocar, DB/Redis com senha ``jefrey`` (pública no GitHub) são acessíveis. `n8n` webhooks sem auth permitem injetar eventos falsos no Jeffrey. ``SECRET_KEY`` no disco, se vazado, compromete todos os ``Bearer`` tokens.

Sugestão de correção: 1) Rotacionar ``SECRET_KEY`` e ``REDIS_PASSWORD`` (gerar ``secrets.token_hex(32)``) 2) Remover ``:-jefrey`` de `docker-compose` → ``${JEFREY_DATABASE__PASSWORD:?missing}`` (fail-closed) 3) ``N8N_BASIC_AUTH_ACTIVE=true`` + gerar ``N8N_BASIC_AUTH_PASSWORD`` forte no setup 4) ``verify_env.py`` falhar se ``password=="jefrey"`` mesmo em DEBUG, a menos que ``JEFREY_ENV=dev`` 5) Adicionar ``pre-commit`` hook ``detect-secrets`` + ``git-secrets``

Critérios de aceite:

- [] ``docker compose config`` sem `.env`` falha (missing var) em vez de usar default
- [] `n8n` UI exige Basic Auth
- [] `verify_env` falha com ``jefrey`` default mesmo com `DEBUG=true` (ou exige `ENV=dev`)
- [] `.env.bak.*`` já ignorado em `.gitignore` (ok)
- [] Nenhum ``:-`` fallback para senha em ``grep -n "\:-" docker-compose.yml``

ISSUE 8

Título: [Segurança] Chaves expostas e defaults inseguros (`docker-compose`, `.env`, `n8n`)
Labels: security, alta, chaves-expostas, p1

Descrição

Múltiplas credenciais com defaults públicos ou expostas no disco/workdir. Agrupado numa issue única por ser mesmo tema.

Evidência

`.env:146` `JEFREY_API__SECRET_KEY=7da4ec1a...946` (real 64 hex) + .env:58` `JEFREY_REDIS__PASSWORD=jefrey_redis_2026` (real fraco) → em disco, não versionado mas .env.bak.202608*` também contém.
`docker-compose.yml:15` `POSTGRES_PASSWORD: ${JEFREY_DATABASE__PASSWORD:-jefrey}` fallback público.
`docker-compose.yml:139` `N8N_BASIC_AUTH_ACTIVE: "false"` → n8n sem auth.
`scripts/setup.py` gera .env` com `jefrey` defaults em modo dev.`

Impacto

****Alta****. Se `.env`` for copiado para prod sem trocar, DB/Redis com senha ``jefrey`` (pública no GitHub) são acessíveis. `n8n` webhooks sem auth permitem injetar eventos falsos no Jeffrey. ``SECRET_KEY`` no disco, se vazado, compromete todos os ``Bearer`` tokens.

Sugestão de correção

- 1) Rotacionar ``SECRET_KEY`` e ``REDIS_PASSWORD`` (gerar ``secrets.token_hex(32)``)
- 2) Remover ``:-jefrey`` de `docker-compose` → ``${JEFREY_DATABASE__PASSWORD:?missing}`` (fail-closed)
- 3) ``N8N_BASIC_AUTH_ACTIVE=true`` + gerar ``N8N_BASIC_AUTH_PASSWORD`` forte no setup
- 4) ``verify_env.py`` falhar se ``password=="jefrey"`` mesmo em DEBUG, a menos que ``JEFREY_ENV=dev``
- 5) Adicionar ``pre-commit`` hook ``detect-secrets`` + ``git-secrets``

Critérios de aceite

- [] ``docker compose config`` sem `.env`` falha (missing var) em vez de usar default
- [] `n8n` UI exige Basic Auth
- [] `verify_env` falha com ``jefrey`` default mesmo com `DEBUG=true` (ou exige `ENV=dev`)
- [] `.env.bak.*`` já ignorado em `.gitignore` (ok)
- [] Nenhum ``:-`` fallback para senha em ``grep -n "\:-" docker-compose.yml``

--- FIM ISSUE 8 ---

ISSUE #9 — [Segurança] Assinaturas frágeis e stubs inseguros (eventbus + token_refresh + jwks + auth middleware inalcançável)

Labels sugeridas: security, média, lógica, p3

Descrição: Quatro dívidas técnicas de criptografia/robustez que não são exploráveis diretamente em prod hoje mas viram vulnerabilidade quando feature for ativada ou em audit.

Evidência: `src/jefrey/eventbus/signing.py:85` canonical_str = str({k: signed[k] for k in canonical_keys})` → str(dict) não determinístico. src/jefrey/oauth2/token_refresh.py:83` if not refresh_token.startswith("valid_")` → stub aceita qualquer prefixo.
src/jefrey/oauth2/jwks.py:128` generate_jwsk_keys()` a cada 24h sem persistir → JWKS efêmero, introspect nunca usa. src/jefrey/api/auth_middleware.py:124` Step 3: Fallback` após `except`:`

```
return 503` → inalcançável.
```

Impacto: ****Média****. Assinatura pode falhar entre Python 3.11/3.12 (repr muda). Stub pode ir para prod por engano e bypassar refresh. JWKS inutilizado. Fallback inalcançável confunde debug (sempre 503 em vez de 401). Nenhum é bypass direto hoje.

Sugestão de correção: 1) `json.dumps(canonical, sort\_keys=True, separators=(",", ":"))` 2) `token\_refresh.py` → se `JEFREY\_OAUTH\_\_TOKEN\_URI` ausente, levantar `NotImplementedError` em vez de stub 3) Persistir JWKS em `config/jwks.json` 00600 e carregar no startup 4) Reordenar auth_middleware: `try: introspect... except: return 503; fallback 401 fora do try`

Critérios de aceite:

- [] `sign\_message` + `verify\_message` passam com keys reordenadas
- [] `refresh\_access\_token("invalid")` → error, não stub
- [] JWKS carregado de arquivo se existir
- [] Auth middleware com token inválido → 401 (não 503) quando introspect retorna inactive
- [] Fallback 401 alcançável (teste com token malformado)

ISSUE 9

Título: [Segurança] Assinaturas frágeis e stubs inseguros (eventbus + token_refresh + jwks + auth middleware inalcançável)

Labels: security, média, lógica, p3

****Descrição****

Quatro dívidas técnicas de criptografia/robustez que não são exploráveis diretamente em prod hoje mas viram vulnerabilidade quando feature for ativada ou em audit.

****Evidência****

src/jefrey/eventbus/signing.py:85 `canonical\_str = str({k: signed[k] for k in canonical\_keys})` → str(dict) não determinístico.

src/jefrey/oauth2/token_refresh.py:83 `if not refresh\_token.startswith("valid\_")` → stub aceita qualquer prefixo.

src/jefrey/oauth2/jwks.py:128 `generate\_jwsk\_keys()` a cada 24h sem persistir → JWKS efêmero, introspect nunca usa.

src/jefrey/api/auth_middleware.py:124 `Step 3: Fallback` após `except: return 503` → inalcançável.

****Impacto****

****Média****. Assinatura pode falhar entre Python 3.11/3.12 (repr muda). Stub pode ir para prod por engano e bypassar refresh. JWKS inutilizado. Fallback inalcançável confunde debug (sempre 503 em vez de 401). Nenhum é bypass direto hoje.

****Sugestão de correção****

- 1) `json.dumps(canonical, sort\_keys=True, separators=(",", ":"))`
- 2) `token\_refresh.py` → se `JEFREY\_OAUTH\_\_TOKEN\_URI` ausente, levantar `NotImplementedError` em vez de stub
- 3) Persistir JWKS em `config/jwks.json` 00600 e carregar no startup
- 4) Reordenar auth_middleware: `try: introspect... except: return 503; fallback 401 fora do try`

****Critérios de aceite****

- [] `sign\_message` + `verify\_message` passam com keys reordenadas
- [] `refresh\_access\_token("invalid")` → error, não stub
- [] JWKS carregado de arquivo se existir
- [] Auth middleware com token inválido → 401 (não 503) quando introspect retorna inactive
- [] Fallback 401 alcançável (teste com token malformado)

--- FIM ISSUE 9 ---